10.11 What to Monitor: Golden Signals, RED and USE

Topic 11 of 11 | ~4 hours | Difficulty: Intermediate

1 What it is

You can collect ten thousand metrics and still not know whether users are happy. This topic is about **choosing**: a small, proven set of signals that answers "is the service working, and if not, where is the problem?" Three checklists do most of the work:

Method	Origin	Applies to	Signals
Four Golden Signals	Google's *Site Reliability Engineering* book	Any user-facing system	**Latency**, **Traffic**, **Errors**, **Saturation**
RED	Tom Wilkie, for microservices	Request-driven services	**Rate**, **Errors**, **Duration**
USE	Brendan Gregg, for performance analysis	Resources (CPU, memory, disk, network, connection pools, queues)	**Utilization**, **Saturation**, **Errors**

They fit together as two layers. **RED (or the golden signals) describes the experience the service delivers**: how many requests, how many fail, how long they take. **USE describes the machinery underneath**: for every resource, how busy it is (utilization), how much work is waiting for it (saturation) and whether it reports faults (errors). Alert on the first layer, because it is what users feel. Use the second layer to find the cause.

Definitions you need:

- **Utilization:** the fraction of time (or capacity) a resource is busy, for example 85 percent CPU.
- **Saturation:** extra work the resource cannot serve yet, waiting in a queue: run-queue length, requests waiting for a database connection, messages backed up in a queue, swap activity. Saturation is usually what hurts latency.
- **Errors:** discrete failure events: HTTP 5xx, disk I/O errors, network packet errors, out-of-memory (OOM) kills.
- **Latency:** time to serve a request. Measure **successful and failed requests separately**: a fast error can make an average look healthy.

2 Why it matters in DevOps

- **Alert fatigue comes from monitoring causes instead of symptoms.** "CPU is at 85 percent" wakes someone for a machine that is fine. "Checkout error ratio above 5 percent for 5 minutes" wakes someone for a real problem. The golden signals are the basis for good alerts and for service level indicators (SLIs §14.1).
- **Dashboards should answer questions in order.** Is it broken (RED)? Where (which route, which instance)? Why (USE on the resources and dependencies)? A dashboard with 60 unordered panels answers none of them.
- **Saturation is the early warning.** Utilization at 70 percent may be fine; a connection pool with 20 waiting requests is already an outage in progress. Teams that only watch utilization are surprised.
- **Every resource needs coverage.** The outage is usually in the one resource nobody graphed: inodes, the connection pool, the queue consumer, the conntrack table.
- **Cost.** Every series costs memory and money. A deliberate list of signals, with bounded labels, keeps the bill and the noise down.

3 Real-world system example

Google's SRE book (the chapter "Monitoring Distributed Systems") names the four golden signals and recommends paging on symptoms that users feel, with cause-oriented metrics kept for diagnosis. Tom Wilkie presented the RED method while working on microservices monitoring at Weaveworks (he later joined Grafana Labs), to give teams one consistent, low-effort dashboard for every service. Brendan Gregg, a performance engineer known for his work at Netflix, published the USE method as a checklist for finding bottlenecks in any system by checking every resource in turn.

These ideas are baked into tooling. The Kubernetes ecosystem's `kube-prometheus-stack` Helm chart ships dashboards and alerts built from node-exporter, cAdvisor and kube-state-metrics, organised around the same resource and workload views; the "monitoring mixins" community project defines such dashboards and alerts as code. Cloud providers expose the same categories: for a managed database, Amazon RDS publishes `CPUUtilization`, `FreeableMemory`, `DatabaseConnections` and disk metrics to CloudWatch (CloudWatch §6.18), and Amazon SQS publishes queue depth (`ApproximateNumberOfMessagesVisible`) and the age of the oldest message.

4 Diagram

Two layers of signals, and the order in which you read them during an incident:

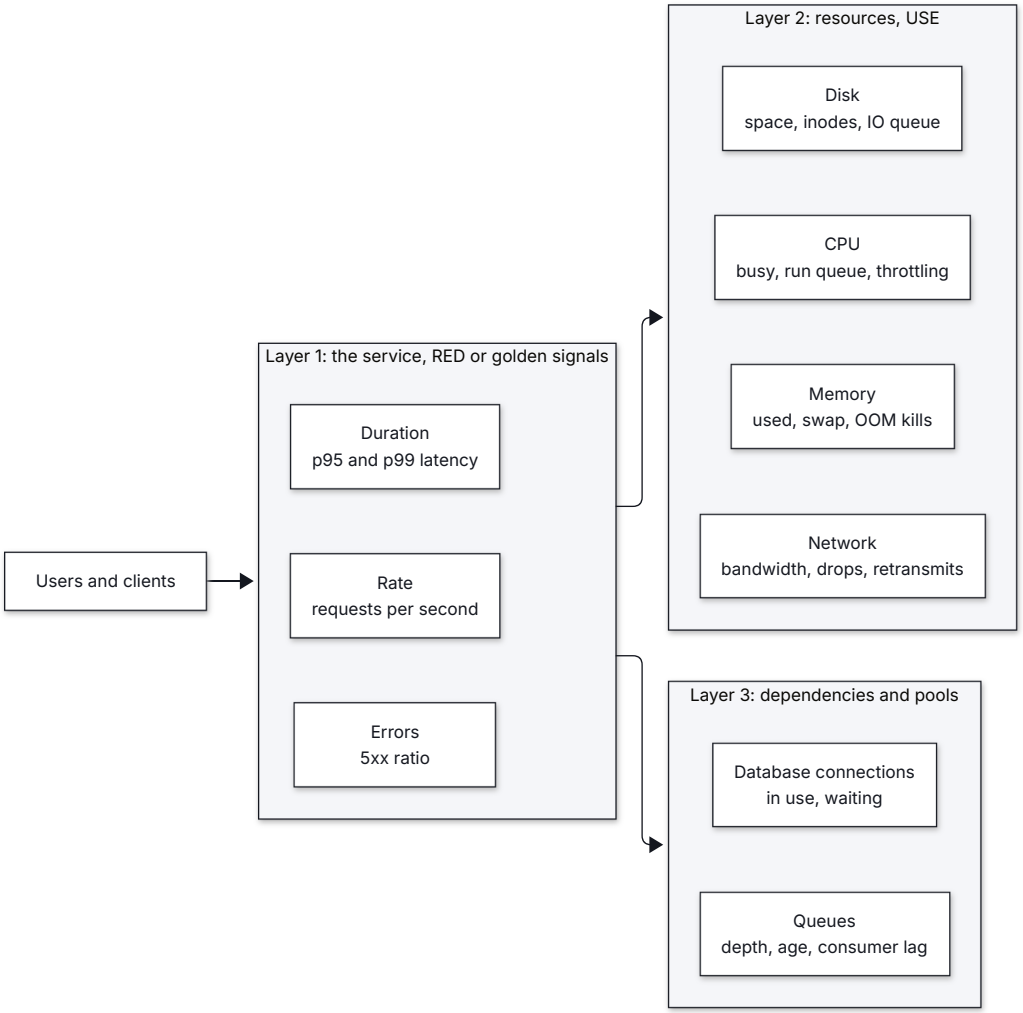


Figure 10.11-1

Alert on the top layer. Investigate down the layers:

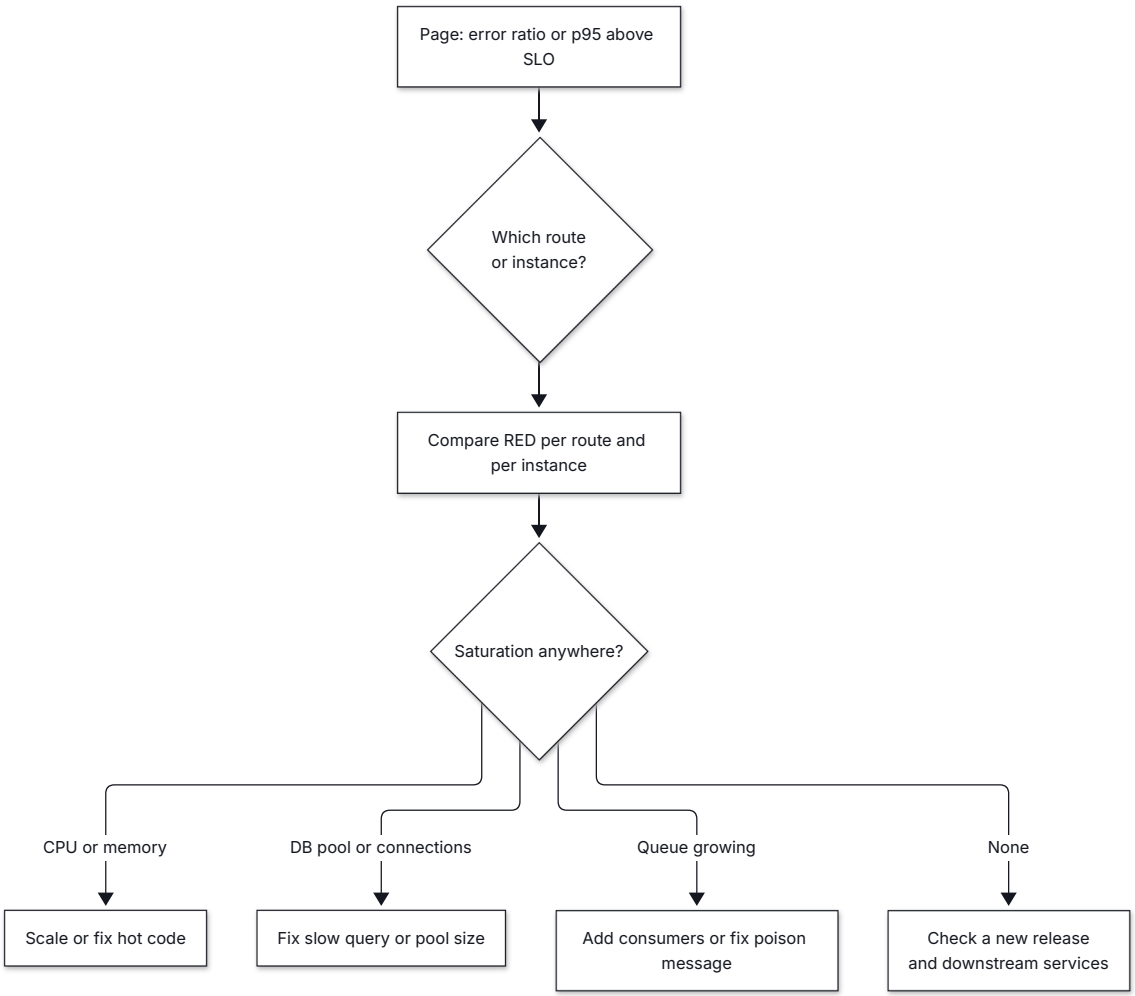


Figure 10.11-2

5 How it works

5.1 The service layer: rate, errors, duration

With the histogram `http_request_duration_seconds` from Prometheus and PromQL §10.2 and Counters, Gauges, Histograms and Summaries §10.4, one metric yields all three signals.

Signal	PromQL	Notes
Request rate (traffic)	`sum by (route) (rate(http_request_duration_seconds_count[5m]))`	Requests per second per route
Error rate	`sum(rate(http_request_duration_seconds_count{status_code=~"5.."}[5m])) / sum(rate(http_request_duration_seconds_count[5m]))`	Ratio is better than a count: it is comparable across traffic levels
Latency p95	`histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))`	Aggregate buckets first, then take the quantile; never average percentiles
Latency, successes only	`histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket{status_code!~"5.."}[5m])))`	Fast failures hide slow successes in a mixed number
Saturation of the process	`nodejs_eventloop_lag_p99_seconds`	Event loop lag is Node.js's own saturation signal

Treat 4xx carefully. A burst of 404s or 401s is often a client or attack problem, not a server fault, so the usual definition of "error" is 5xx plus responses that are wrong in other ways (a 200 that carries an error body, a response slower than the SLO).

Use **route templates** (`/orders/:id`), never raw URLs, as label values. Otherwise one label explodes into millions of series.

5.2 The resource layer: USE for each resource

Run through this table for every host or node. On Kubernetes the same ideas apply to nodes and, with cAdvisor metrics, to containers.

Resource	Utilization	Saturation	Errors
CPU	`1 - avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m]))`	Run queue per core: `node_load1 / count by (instance) (node_cpu_seconds_total{mode="idle"})` . Better, the pressure-stall metric: `rate(node_pressure_cpu_waiting_seconds_total[5m])` (Linux 4.20+). In containers: CPU throttling	Rare: machine-check errors

Resource	Utilization	Saturation	Errors
Memory	`1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes`	Swap activity `rate(node_vmstat_pswpin[5m]) + rate(node_vmstat_pswpout[5m])`, major faults `rate(node_vmstat_pgmajfault[5m])`	OOM kills: `increase(node_vmstat_oom_kill[1h])`
Disk space	`1 - node_filesystem_avail_bytes{fstype!~"tmpfs|overlay|squashfs"} / node_filesystem_size_bytes{fstype!~"tmpfs|overlay|squashfs"}`	**Predict it**: `predict_linear(node_filesystem_avail_bytes{fstype!~"tmpfs|overlay|squashfs"}[6h], 4 * 3600) < 0`	Read-only filesystem: `node_filesystem_readonly == 1`. Also watch **inodes**: `node_filesystem_files_free / node_filesystem_files`
Disk I/O	`rate(node_disk_io_time_seconds_total[5m])` (fraction of time busy)	Average queue length: `rate(node_disk_io_time_weighted_seconds_total[5m])`	Device errors in the kernel log and SMART data
Network	`rate(node_network_receive_bytes_total{device!~"lo|veth.*|br-.*|docker.*"}[5m]) * 8` (bits per second) against the link speed	Dropped packets `rate(node_network_receive_drop_total[5m])`; TCP retransmits `rate(node_netstat_Tcp_RetransSegs[5m]) / rate(node_netstat_Tcp_OutSegs[5m])`; the conntrack table `node_nf_conntrack_entries / node_nf_conntrack_entries_limit`	Packet errors `rate(node_network_receive_errs_total[5m])`

Metric names come from node_exporter's default collectors; the pressure and conntrack metrics exist only if the kernel exposes them (Docker Desktop's virtual machine may not). Check `/metrics` before building an alert on a name. Command-line equivalents for the same checks are in System Monitoring Commands §1.10.

Two classic traps:

- **Average CPU hides a pinned core.** A single-threaded process (Node.js event loop) can use 100 percent of one core while the host shows 12 percent. Look at the process or per-core values, and at event loop lag.
- **Memory "used" includes cache.** Use `MemAvailable`, not "free". High page cache is healthy; swapping and OOM kills are not.

5.3 Dependencies: database connections and queues

Pools and queues are resources too, with the same three questions.

Database connections. Each PostgreSQL connection costs memory and a server process, so the server has a hard limit (`max_connections`, 100 by default) and your application usually has a pool in front of it (Database Connection Pooling §13.6).

USE	App-side (pool, exported by your code)	Server-side (PostgreSQL exporter)
Utilization	`(db_pool_connections{state="total"} - db_pool_connections{state="idle"}) / db_pool_max_connections`	`sum(pg_stat_activity_count) / max(pg_settings_max_connections)`
Saturation	`db_pool_connections{state="waiting"}`: requests queued for a connection	Sessions in state `idle in transaction`, or connection attempts that fail
Errors	Pool timeouts and `too many clients` errors in logs and a counter	`pg_stat_database_xact_rollback`, deadlocks `pg_stat_database_deadlocks`

The app-side gauges are easy to export with the `pg` library's pool and `prom-client`:

```JS
const pool = new Pool({ max: 10, connectionTimeoutMillis: 2000 });
new client.Gauge({
  name: 'db_pool_connections', help: 'Pool connections by state', labelNames: ['state'],
  collect() {
    this.set({ state: 'total' }, pool.totalCount);
    this.set({ state: 'idle' }, pool.idleCount);
    this.set({ state: 'waiting' }, pool.waitingCount);
  },
});
new client.Gauge({ name: 'db_pool_max_connections', help: 'Configured pool size'
  }).set(10);
```

Exporter metric names differ slightly between exporter versions and exporters, so confirm them on the exporter's `/metrics` page. Managed databases publish an equivalent (RDS: `DatabaseConnections` in CloudWatch).

Queues. A queue converts a spike into a delay, which is only acceptable if consumers catch up. Monitor four things:

Signal	Meaning	Examples
Depth	Messages waiting	Redis list length (`redis_key_size`), RabbitMQ ready messages, SQS visible messages
Age of the oldest message	How late the work is: this is the user-visible latency	SQS `ApproximateAgeOfOldestMessage`, a timestamp in your own messages
Consumer lag (Kafka)	How far a consumer group is behind the log	Kafka exporters expose consumer-group lag per partition (Kafka §13.8)

Signal	Meaning	Examples
Throughput in and out	Arrival rate versus processing rate: depth grows when in is greater than out	Counters on enqueue and on completion

Depth alone misleads (a queue with 10,000 messages and 5,000 per second drain is fine; 50 messages and a stuck consumer is not). Alert on "depth is growing for 10 minutes" or "oldest message older than N minutes". Also watch the **dead-letter queue**: any message there is an error (RabbitMQ §13.9).

5.4 Turning signals into alerts

Layer	Alert on	Typical response
Symptom (RED)	Error ratio above threshold for several minutes, p95 above the SLO, traffic suddenly zero (`absent` or a ratio to last week)	Page
Imminent resource exhaustion	`predict_linear` says the disk fills within hours, DB pool 90 percent used for 10 minutes, queue growing	Ticket or low-urgency notification, page if close
Causes (CPU 80 percent, memory 70 percent)	Almost never alert; graph them	Dashboard only

Guard ratio alerts against low traffic: one failing request out of two is a 50 percent error ratio. Add a minimum request-rate condition. Keep alert counts low; see Alert Fatigue Reduction §14.8.

6 Step-by-step guide

Prerequisites: the Compose project `~/l10-prom` from Prometheus and PromQL §10.2 running, with `shop-api`, the load generator, node-exporter and Prometheus on port 9090. Docker with Compose v2, about 1 GB of free RAM. No cost. Image tags were current when written; if one is missing, use the newest tag and read its notes. Some host metrics (pressure stall, conntrack) are absent on Docker Desktop.

1. **Add PostgreSQL, Redis and their exporters** to `~/l10-prom/compose.yaml` under `services:` and add `pg-data: {}` under `volumes:`.

```yaml
  postgres:
    image: postgres:17-alpine
    environment:
      POSTGRES_PASSWORD: postgres        # lab only
    volumes: ["pg-data:/var/lib/postgresql/data"]
```

```
  postgres-exporter:
    image: quay.io/prometheuscommunity/postgres-exporter:v0.17.1
    environment:
      DATA_SOURCE_NAME: "postgresql://postgres:postgres@postgres:5432/postgres?
  sslmode=disable"
    depends_on: [postgres]

  redis:
    image: redis:7-alpine

  redis-exporter:
    image: oliver006/redis_exporter:v1.67.0
    command: ["--redis.addr=redis://redis:6379", "--check-single-
  keys=db0=queue:orders"]
    depends_on: [redis]
```

`--check-single-keys` makes the exporter report the length of the list `queue:orders`.

2. **Add scrape jobs** to `~/l10-prom/prometheus/prometheus.yml` under `scrape_configs:`.

```yaml
  - job_name: postgres
    static_configs:
      - targets: ["postgres-exporter:9187"]
  - job_name: redis
    static_configs:
      - targets: ["redis-exporter:9121"]
```

3. **Write recording and alerting rules** for both layers in `~/l10-prom/prometheus/rules/golden.yml`.

```yaml
groups:
  - name: golden-recording
    rules:
      - record: job_route:http_requests:rate5m
        expr: sum by (job, route) (rate(http_request_duration_seconds_count[5m]))
      - record: job:http_request_duration_seconds:p95_ok_5m
        expr: |
          histogram_quantile(0.95,
            sum by (job, le)
  (rate(http_request_duration_seconds_bucket{status_code!~"5.."}[5m])))
      - record: instance:node_cpu_utilisation:ratio_rate5m
        expr: 1 - avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m]))
      - record: instance:node_memory_utilisation:ratio
        expr: 1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes

  - name: golden-alerts
    rules:
      - alert: ShopApiLatencyHigh
```

```
        expr: job:http_request_duration_seconds:p95_ok_5m{job="shop-api"} > 0.5
        for: 5m
        labels: { severity: page }
        annotations:
          summary: "shop-api p95 latency is {{ $value | humanizeDuration }}"
      - alert: DiskWillFillSoon
        expr:
  predict_linear(node_filesystem_avail_bytes{fstype!~"tmpfs|overlay|squashfs"}[6h], 4
  * 3600) < 0
        for: 30m
        labels: { severity: ticket }
        annotations:
          summary: "{{ $labels.mountpoint }} on {{ $labels.instance }} is predicted to
  fill within 4 hours"
      - alert: OrderQueueBacklogGrowing
        expr: redis_key_size{key="queue:orders"} > 500 and
  deriv(redis_key_size{key="queue:orders"}[10m]) > 0
        for: 10m
        labels: { severity: ticket }
        annotations:
          summary: "queue:orders has {{ $value }} messages and is still growing"
```

Check the file before loading it, using the same image Prometheus runs from:

```bash
cd ~/l10-prom
docker run --rm --entrypoint promtool -v "$PWD/prometheus/rules:/rules:ro"
  prom/prometheus:v3.15.0 check rules /rules/golden.yml
```

Expect `SUCCESS: 7 rules found`.

4. **Start the new services and reload Prometheus.**

```bash
docker compose up -d
curl -X POST http://localhost:9090/-/reload
sleep 30
```

In http://localhost:9090/targets the `postgres` and `redis` jobs show `UP`. If `postgres` is down, read `docker compose logs postgres-exporter`.

5. **Read the service layer (RED).** In http://localhost:9090/query run the first four queries of section 5.1. For this lab the rate per route is a few requests per second, the 5xx ratio is near 0.04 and `/checkout` p95 is a few hundred milliseconds (similar to; the app is random). Then the recorded versions: `job_route:http_requests:rate5m`.
6. **Create CPU saturation and watch USE.** Start four busy loops (adjust to your core count),

then compare utilization and saturation.

```bash
for i in 1 2 3 4; do docker run -d --rm --name burn$i busybox sh -c 'while :; do :;
  done'; done
```

After a minute, `instance:node_cpu_utilisation:ratio_rate5m` rises toward `1` on a machine with four cores or fewer, and `node_load1 / count by (instance) (node_cpu_seconds_total{mode="idle"})` goes above `1`; on Linux hosts `rate(node_pressure_cpu_waiting_seconds_total[1m])` rises too. Check whether the symptom layer noticed: `shop-api` p95 may grow because requests wait for CPU. Stop the load:

```bash
for i in 1 2 3 4; do docker rm -f burn$i; done
```

7. **Memory and disk.** Run the two memory queries from the table and note that `MemAvailable` is much larger than "free". Then the disk queries:

```promql
1 - node_filesystem_avail_bytes{fstype!~"tmpfs|overlay|squashfs"} /
  node_filesystem_size_bytes{fstype!~"tmpfs|overlay|squashfs"}
predict_linear(node_filesystem_avail_bytes{fstype!~"tmpfs|overlay|squashfs"}[6h], 4 *
  3600)
```

   The second is the forecast of free bytes in four hours; it is negative only if the disk is filling quickly. To see it move, write a 2 GB file in a temporary folder (`fallocate -l 2G /tmp/fill.bin`), query again after a few scrapes, then delete the file.

8. **Network.** Run `sum by (device) (rate(node_network_transmit_bytes_total[5m]))` without the device filter and find the Docker bridge carrying the traffic from `loadgen` to `shop-api`. Run the drop and error queries from the table; they should be zero. Note which devices you would exclude from an alert (`lo`, `veth*`).

9. **Database connections (saturation).** Hold three connections open for two minutes and look at them from both sides.

```bash
for i in 1 2 3; do docker compose exec -d postgres psql -U postgres -c 'select
  pg_sleep(120)'; done
docker compose exec postgres psql -U postgres -c "select state, count(*) from
  pg_stat_activity where datname is not null group by 1"
```

The SQL shows three `active` sessions (plus maybe an `idle` one from the exporter). In Prometheus run `sum by (state) (pg_stat_activity_count)`: the same numbers appear after the next scrape. Compare against `max(pg_settings_max_connections)` if the metric exists in your exporter version (otherwise look at `show max_connections;` in SQL, `100` by default) and compute utilization. The capstone adds the app-side pool gauges from section 5.3.

10. **Queue depth and growth.** Push jobs faster than you consume them.

```bash
docker compose exec redis sh -c 'for i in $(seq 2000); do echo "LPUSH queue:orders
  job$i"; done | redis-cli > /dev/null'
docker compose exec redis redis-cli llen queue:orders
```

Expect `2000`. After a scrape, `redis_key_size{key="queue:orders"}` shows it. Drain part of it and see the slope turn negative:

```bash
docker compose exec redis sh -c 'for i in $(seq 1500); do echo "RPOP queue:orders";
  done | redis-cli > /dev/null'
```

Then `deriv(redis_key_size{key="queue:orders"}[5m])` is negative. Re-push to make the alert `OrderQueueBacklogGrowing` go Pending and Firing at http://localhost:9090/alerts.

11. **Build a USE and RED overview.** In Grafana (http://localhost:3001) create a dashboard `shop-api overview` with two rows: *Service* (request rate by route, 5xx ratio, p95 latency) and *Resources* (CPU utilisation and load per core, memory available, disk fill forecast, DB connections by state, queue depth). Put the symptom row first. Export it as JSON into `grafana/dashboards/` as in Grafana Dashboards §10.3.

12. **Clean up.**

```bash
docker compose down          # add -v to delete stored data
rm -f /tmp/fill.bin
```

7 Real-world scenario

Illustrative scenario. A team's only alert was "CPU above 80 percent for 10 minutes". It fired every night during a batch job (harmless) and never fired during the incident that mattered. On a Tuesday afternoon checkout latency tripled and 3 percent of requests failed. CPU was 35 percent. The dashboard they built the following week, in USE and RED order, would have shown the cause in one glance.

Reading the layers: the *service* row showed p95 up from 180 ms to 2.4 s and the 5xx ratio at 3 percent. In the *resources* row CPU, memory, disk and network all looked normal, but the *dependencies* row showed the database pool at 10 of 10 connections with 40 requests waiting. A new report endpoint ran a 20-second query on the same pool as checkout, taking connections away. The fix was a separate small pool with a statement timeout for reports, and an index for the query.

Lesson: utilization of CPU was irrelevant; **saturation** of a pool was the cause. They replaced the CPU alert with an error ratio and p95 alert (symptoms) and a ticket-level alert on "pool waiting greater than 0 for 10 minutes" (early warning).

8 Common mistakes and best practices

Don't

- Page on CPU, memory or load alone. Graph them; page on user-visible symptoms.
- Average latency, or average percentiles across instances. Use histograms and aggregate buckets.
- Count errors instead of ratios: a count alert fires on busy days and misses quiet-hour outages.
- Forget saturation: pools, queues, thread pools and file descriptors fail at 100 percent while the host looks idle.
- Alert on one scrape. Use `for:` durations and a minimum traffic condition so a single failed request does not page.
- Use unbounded labels (user ID, raw URL, request ID) on metrics.
- Ignore disk inodes, certificate expiry and the dead-letter queue because they are not on the host dashboard.
- Build a new dashboard layout for every service. Consistency lets any engineer read any service.

Do

- Give every service the same RED panels and every host/node the same USE panels; templating by `job` and `instance` makes this cheap.
- Alert on symptoms (error ratio, p95 against the SLO), with forecasts (`predict_linear`) for resources that run out.
- Monitor each dependency from both sides: your client's view (pool, timeouts, retries) and the server's view.
- Record expensive queries as recording rules and use them in dashboards and alerts.

- Record what you decided: for each alert, a runbook link and the first dashboard panel to open.
- Review alerts after incidents: add the missing signal, delete the alert that never helped.

9 Tasks

- [ ] **T1 — Build the RED panels for a service** *(≈45 min, beginner)*
  - **Goal:** produce the three service-layer signals from one histogram.
  - **Steps:**
    1. Run the lab through step 5.
    2. In Grafana add three panels: request rate by route, 5xx ratio, p95 latency (use `$__rate_interval` instead of `5m`).
    3. Add a `route` variable and confirm the panels change.
  - **Done when:** all three panels show data and the error ratio panel is near 4 percent.
- [ ] **T2 — Walk the USE table on your machine** *(≈60 min, intermediate)*
  - **Goal:** fill a USE sheet with real numbers.
  - **Steps:**
    1. For CPU, memory, disk, disk I/O and network write down the utilization, saturation and error query, and the current value.
    2. Mark every cell where your system lacks a metric (for example pressure stall data) and say how you would get one.
    3. Compare the CPU row with `uptime`, `free -m` and `df -h` (System Monitoring Commands §1.10).
  - **Done when:** the sheet has a value or "n/a, because ..." in every cell, and the numbers agree with the command-line tools.
- [ ] **T3 — Make saturation visible** *(≈45 min, intermediate)*
  - **Goal:** show that high utilization and saturation are different.
  - **Steps:**
    1. Run step 6 with two busy loops, then with eight, on a machine with four cores.
    2. Record CPU utilization, load per core and `shop-api` p95 each time.
    3. Explain which metric changed between the two runs and which one predicts latency.
  - **Done when:** you have two rows of numbers and a written explanation that utilization stays near 100 percent while saturation and latency keep rising.

- [ ] **T4 — Break the database pool and find it from the signals** *(≈60 min, advanced)*
  - **Goal:** diagnose pool exhaustion from metrics.
  - **Steps:**
    1. Add a `pg` pool of size 5 and the `db_pool_connections` gauges (section 5.3) to a route in `shop-api`.
    2. Make one route hold a connection for 3 seconds (`select pg_sleep(3)`) and call it ten times in parallel.
    3. Watch `waiting`, the p95 latency of an unrelated route and the 5xx ratio.
  - **Done when:** you can point to the `waiting` gauge rising before the error ratio, and an alert on `waiting > 0 for 5m` fires.

- [ ] **T5 — Queue alert with a test** *(≈45 min, advanced)*
  - **Goal:** alert on a growing backlog, not on depth alone.
  - **Steps:**
    1. Do step 10, pushing 2000 jobs then draining 1500.
    2. Confirm `OrderQueueBacklogGrowing` does not fire when depth is above 500 but shrinking.
    3. Push more jobs without draining and confirm it fires after the `for` period. Add a rule for `oldest job age` if your jobs carry a timestamp.
  - **Done when:** the alert fires only when the queue is both large and growing, and you can show both cases in the Prometheus alerts page.

- [ ] **T6 — Prove it: design the monitoring plan** *(≈60 min, advanced)*
  - **Goal:** choose signals and alerts for a service without a template.
  - **Steps:**
    1. Without the guide, write the plan for `shop-api` + PostgreSQL + Redis queue: the RED metrics, the USE entry for every resource and dependency, and which three become alerts.
    2. For each alert write: expression, `for` duration, severity, the first dashboard panel to open and the likely action.
    3. Explain why you do not page on CPU utilization.
  - **Done when:** a teammate can build the dashboard and the alerts from your plan and the plan covers CPU, memory, disk, network, rate, errors, latency, DB connections and queues.

10 Self-check questions

1. Name the four golden signals, and say how RED relates to them.
2. What is the difference between utilization and saturation? Give an example where one is low and the other high.
3. Why measure latency of successful and failed requests separately?
4. Which signals describe a database connection pool, and which one would you alert on first?
5. Why is `predict_linear` better than a "disk above 90 percent" alert for some disks?
6. For a queue, why is depth alone a poor signal?

ANSWERS

1. Latency, traffic, errors and saturation. RED (rate, errors, duration) is the first three applied to request-driven services; USE covers saturation and the resources.
2. Utilization is how busy a resource is; saturation is the work that cannot be served yet and waits. A database pool can be 100 percent used (high utilization) with nothing waiting, or a single-threaded process can use one core fully (host CPU at 12 percent) while requests queue behind it.
3. Failures are often fast (an immediate 500) and make mixed percentiles look better than what successful users see; or slow because of timeouts. Separating them shows both user experiences.
4. Utilization (in-use over max), saturation (requests waiting for a connection) and errors (timeouts, `too many clients`). Alert first on sustained waiting, because it appears before errors and before the pool is fully exhausted.
5. A 1 TB disk at 90 percent has 100 GB left and may be fine for weeks, while a 20 GB disk at 70 percent can fill in an hour. A forecast warns based on how fast space is being used, with time to react.
6. A large depth can be normal (a batch) and a small depth can hide a stuck consumer. Depth trend, the age of the oldest message, consumer lag and the in/out rate show whether work is being completed in time.

11 Further reading

- *Site Reliability Engineering* (Google, free at sre.google/books): the chapter "Monitoring Distributed Systems" for the four golden signals.
- Brendan Gregg, "The USE Method" (brendangregg.com) and *Systems Performance*.
- Tom Wilkie, "The RED Method" (Weaveworks and Grafana Labs blog posts and talks).

- Prometheus documentation: "Alerting best practices" and the node_exporter and Alertmanager guides.
- The `kubernetes-monitoring/kubernetes-mixin` and `monitoring.mixins.dev` projects, and the `kube-prometheus-stack` chart documentation, for ready-made dashboards and alerts.
- See also: Service Level Indicators §14.1, Prometheus and PromQL §10.2, Grafana Dashboards §10.3, Sentry §10.10.